

Setting up a Proxmox home virtual environment for some virtual machine migrations and creations

Creation of Containers/Kubernetes Cluster – Proof of concepts

3 node setup:

nodes: 8gb ram, 4 cpu, 50gb storage with debian 13

Test configurations:

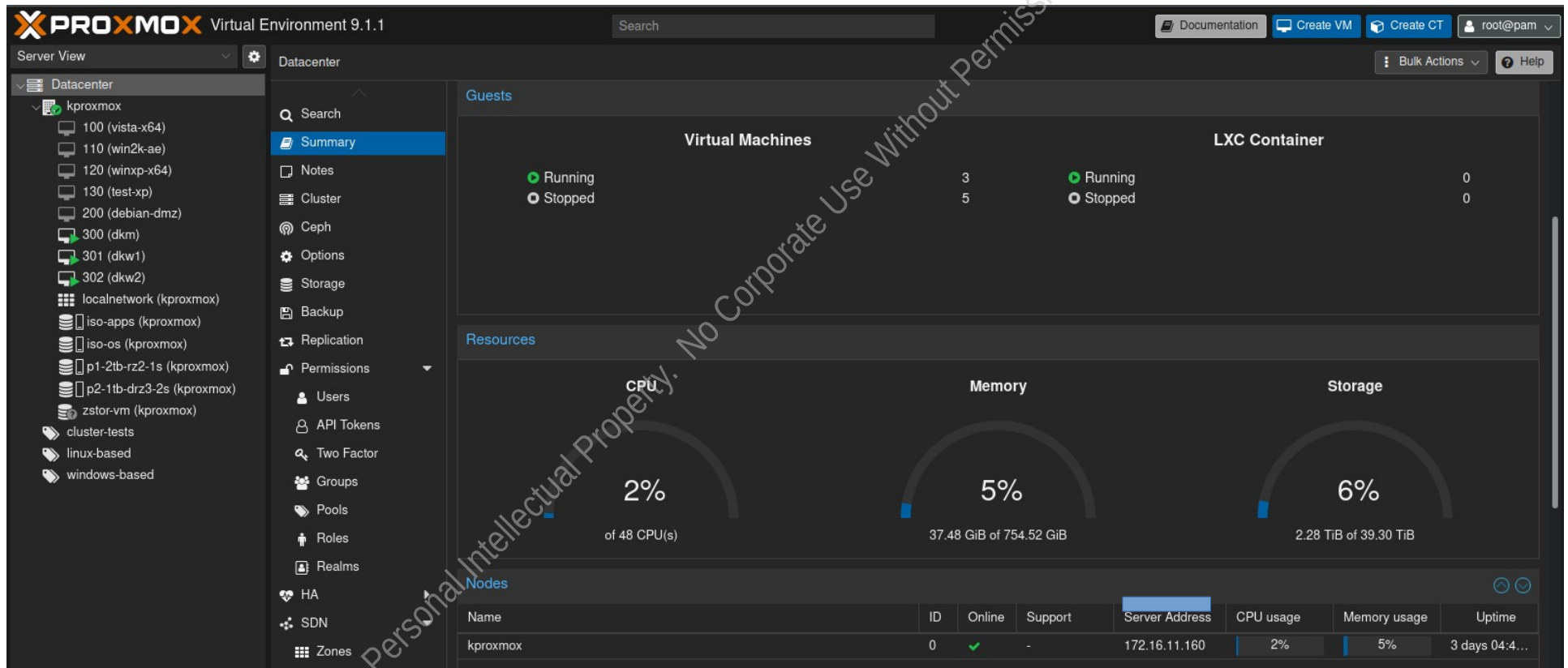
Creation of the nodes

Base configurations for joining the cluster

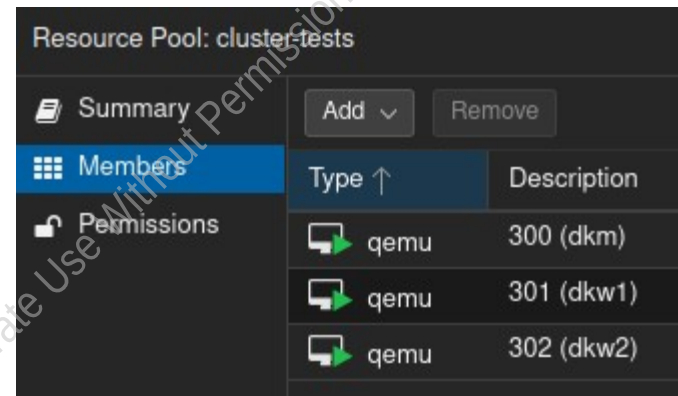
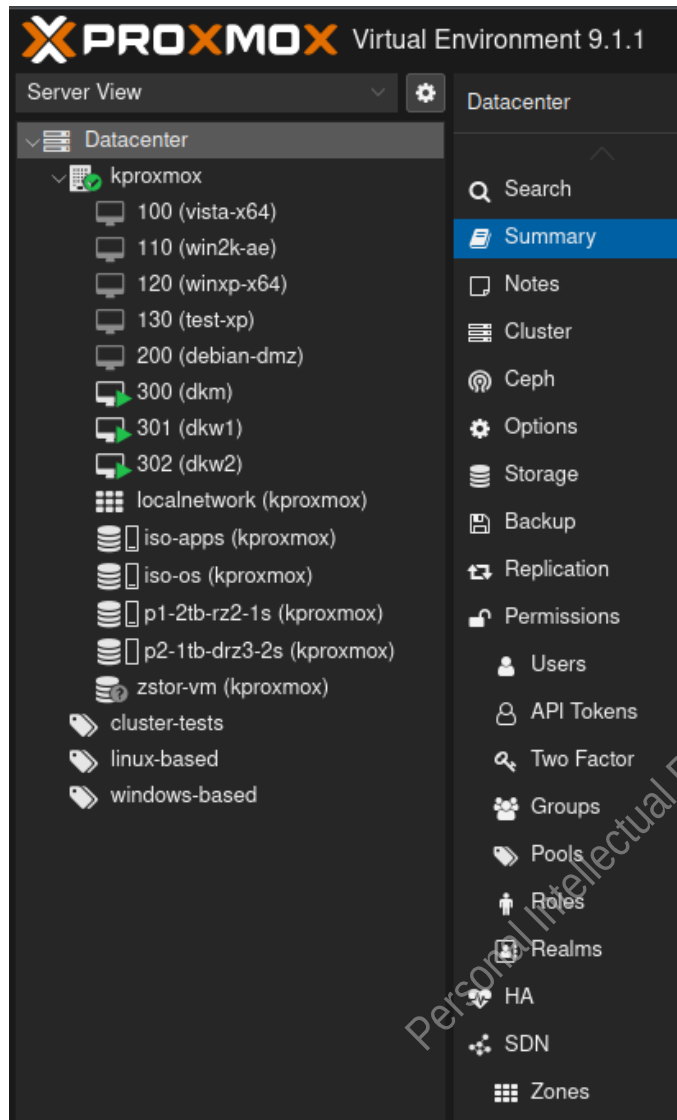
Proof of concept workload for nginx and a small ai language model

Incremental/iterative build and expansion of features and configurations – ongoing progress

// Here's the broad environments (dynamic – vms added/deleted/migrated, ongoing progress)



// Expanded view



Disk usage...	Memory us...	CPU usage	Uptime	Host CPU ...	Host Mem...
0.0 %	94.8 %	23.0% of 4 ...	17:58:07	1.9% of 48 ...	1.1 %
0.0 %	19.2 %	2.9% of 4 ...	02:36:48	0.2% of 48 ...	0.2 %
0.0 %	34.4 %	1.9% of 4 ...	01:59:35	0.2% of 48 ...	0.4 %

Datacenter

Search Add Remove Edit

Summary

Notes

Cluster

Ceph

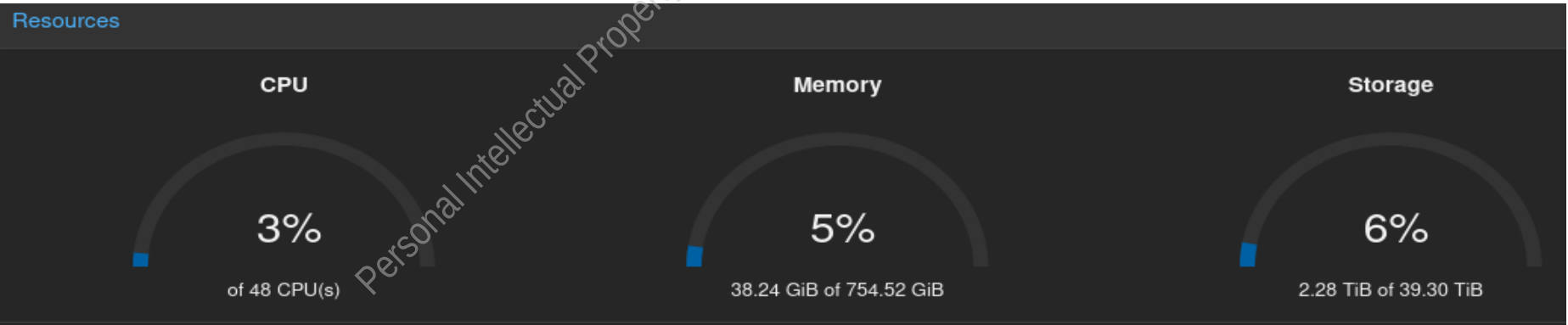
Options

Storage

Backup

Replication

ID ↑	Type	Content	Path/Target
iso-apps	Directory	Backup, Disk image, Import, ISO image, Co...	/p1-2tb-rz2-1s/iso-apps
iso-os	Directory	Backup, Disk image, Import, ISO image, Co...	/p1-2tb-rz2-1s/iso-os
local	Directory	Backup, Import, ISO image, Container temp...	/var/lib/vz
local-lvm	LVM-Thin	Disk image, Container	
p1-2tb-rz2-1s	ZFS	Disk image, Container	
p2-1tb-drz3-2s	ZFS	Disk image, Container	
zstor-vm	NFS	Disk image	/mnt/pve/zstor-vm



// Initial setup and test of the kubernetes cluster using Debian, additional test clusters eventually to be experimented with :
Rocky Linux and Suse Server 16 (along with ceph storage)

// Getting the gpg key for the kubernetes/k8s repo

```
root@dkm:/home/test# wget https://pkgs.k8s.io/core:/stable:/v1.35/deb/Release.key
--2026-02-02 01:45:51-- https://pkgs.k8s.io/core:/stable:/v1.35/deb/Release.key
Resolving pkgs.k8s.io (pkgs.k8s.io)... 34.107.204.206, 2600:1901:0:26f3::
Connecting to pkgs.k8s.io (pkgs.k8s.io)|34.107.204.206|:443... connected.
HTTP request sent, awaiting response... 302 Found
Location: https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.35/deb/Release.key [following]
--2026-02-02 01:45:52-- https://prod-cdn.packages.k8s.io/repositories/isv:/kubernetes:/core:/stable:/v1.35/deb/Release.key
Resolving prod-cdn.packages.k8s.io (prod-cdn.packages.k8s.io)... 13.33.252.97, 13.33.252.46, 13.33.252.37, ...
Connecting to prod-cdn.packages.k8s.io (prod-cdn.packages.k8s.io)|13.33.252.97|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1738 (1.7K) [application/vnd.apple.keynote]
Saving to: 'Release.key'

Release.key                               100%[=====>]
2026-02-02 01:45:54 (11.0 MB/s) - 'Release.key' saved [1738/1738]
```

// adding the gpg key to keyring, updating the repo, and installing the main files (not shown is the containerd, also needed)

```
root@dkm:/home/test# gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg Release.key
root@dkm:/home/test# vi /etc/apt/sources.list
root@dkm:/home/test#
root@dkm:/home/test#
root@dkm:/home/test#
root@dkm:/home/test# apt-get update
Hit:1 http://deb.debian.org/debian trixie InRelease
Hit:2 http://security.debian.org/debian-security trixie-security InRelease
Hit:3 http://deb.debian.org/debian trixie-updates InRelease
Get:4 https://prod-cdn.packages.k8s.io/repositories/isv/kubernetes:/core:/stable:/v1.32/deb InRelease [1,230 B]
Get:5 https://prod-cdn.packages.k8s.io/repositories/isv/kubernetes:/core:/stable:/v1.32/deb Packages [16.1 kB]
Fetched 17.4 kB in 2s (9,451 B/s)
Reading package lists... Done
root@dkm:/home/test#
root@dkm:/home/test#
root@dkm:/home/test# apt-get upgrade
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@dkm:/home/test# apt-get install kubeadm kubelet kubectl
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following additional packages will be installed:
```

// created and updated some config files (not inclusive of all configurations displayed)

```
root@dkm:/home/test# vi /etc/modules-load.d/kubernetes.conf
root@dkm:/home/test# vi /root/.bashrc
root@dkm:/home/test#
root@dkm:/home/test# pwd
/home/test
root@dkm:/home/test# vi Documents/dkm-config.yaml
root@dkm:/home/test# vi /etc/sysctl.d/kubernetes.conf
root@dkm:/home/test# ls /etc/con
console-setup/ containerd/
root@dkm:/home/test# ls /etc/con
console-setup/ containerd/
root@dkm:/home/test# cat /etc/containerd/config.toml
version = 2

[plugins]
  [plugins."io.containerd.grpc.v1.cri"]
    [plugins."io.containerd.grpc.v1.cri".cni]
      bin_dir = "/usr/lib/cni"
      conf_dir = "/etc/cni/net.d"
    [plugins."io.containerd.internal.v1.opt"]
      path = "/var/lib/containerd/opt"
root@dkm:/home/test#
root@dkm:/home/test#
root@dkm:/home/test#
root@dkm:/home/test#
root@dkm:/home/test# containerd config default
```

// started the initialization of the kubernetes cluster (on the master/control plane)

```
root@dkm:/home/test# kubeadm init --config Documents/dkm-config.yaml
[init] Using Kubernetes version: v1.35.0
[preflight] Running pre-flight checks
[WARNING KubernetesVersion]: Kubernetes version is greater than kubeadm version. Please consider to upgrade kubeadm version: 1.32.x
[WARNING FileExisting-ethtool]: ethtool not found in system path
[preflight] Pulling images required for setting up a Kubernetes cluster
[preflight] This might take a minute or two, depending on the speed of your internet connection
[preflight] You can also perform this action beforehand using 'kubeadm config images pull'
W0202 02:20:37.673819 6354 checks.go:843] detected that the sandbox image "registry.k8s.io/pause:3.8" of the CRI is deprecated by kubeadm. It is recommended to use "registry.k8s.io/pause:3.10" as the CRI sandbox image.
[certs] Using certificateDir folder "/etc/kubernetes/pki"
[certs] Generating "ca" certificate and key
[certs] Generating "apiserver" certificate and key
[certs] apiserver serving cert is signed for DNS names [dkm kubernetes kubernetes.default kubernetes.default.svc kubernetes.default.svc.cluster.local] and IPs [10.96.0.1 172.16.11.170]
[certs] Generating "apiserver-kubelet-client" certificate and key
[certs] Generating "front-proxy-ca" certificate and key
[certs] Generating "front-proxy-client" certificate and key
[certs] Generating "etcd/ca" certificate and key
[certs] Generating "etcd/server" certificate and key
[certs] etcd/server serving cert is signed for DNS names [dkm localhost] and IPs [10.96.0.1 172.16.11.170]::1
[certs] Generating "etcd/peer" certificate and key
[certs] etcd/peer serving cert is signed for DNS names [dkm localhost] and IPs [10.96.0.1 172.16.11.170]:1
[certs] Generating "etcd/healthcheck-client" certificate and key
[certs] Generating "apiserver-etcd-client" certificate and key
[certs] Generating "sa" key and public key
```


// continued output

```
[bootstrap-token] Configured RBAC rules to allow Node Bootstrap tokens to post CSRs in
[bootstrap-token] Configured RBAC rules to allow the csrapprover controller automatica
[bootstrap-token] Configured RBAC rules to allow certificate rotation for all node cli
[bootstrap-token] Creating the "cluster-info" ConfigMap in the "kube-public" namespace
[kubelet-finalize] Updating "/etc/kubernetes/kubelet.conf" to point to a rotatable kub
[addons] Applied essential addon: CoreDNS
[addons] Applied essential addon: kube-proxy

Your Kubernetes control-plane has initialized successfully!

To start using your cluster, you need to run the following as a regular user:

  mkdir -p $HOME/.kube
  sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
  sudo chown $(id -u):$(id -g) $HOME/.kube/config

Alternatively, if you are the root user, you can run:

  export KUBECONFIG=/etc/kubernetes/admin.conf

You should now deploy a pod network to the cluster.
Run "kubectl apply -f [podnetwork].yaml" with one of the options listed at:
  https://kubernetes.io/docs/concepts/cluster-administration/addons/

You can now join any number of control-plane nodes by copying certificate authorities
and service account keys on each node and then running the following as root:
```

// continued steps post initialization

```
test@dkm:~$ kubectl apply -f https://github.com/flannel-io/flannel/releases/latest/download/kube-flannel.yml
namespace/kube-flannel created
serviceaccount/flannel created
clusterrole.rbac.authorization.k8s.io/flannel created
clusterrolebinding.rbac.authorization.k8s.io/flannel created
configmap/kube-flannel-cfg created
daemonset.apps/kube-flannel-ds created
test@dkm:~$
```

// local master control plane verification

```
test@dkm:~$ kubectl get nodes
NAME     STATUS    ROLES    AGE     VERSION
dkm      Ready     control-plane  8m16s   v1.32.11
```

// built out the first worker vm node, configured, ready to join the cluster

```
root@dkw1:/home/test# kubeadm join [REDACTED]:6443 --token hp01nfj8dg6el173973w4e --discovery-token-ca
0b31b7a3c7344e90585135b41f429b7
[preflight] Running pre-flight checks
[WARNING ContainerRuntimeVersion]: You must update your container runtime to a version that support
ing cgroupDriver from kubelet config will be removed in 1.36. For more information, see https://git.k8s.io
tion-over-cri
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config kubeadm --config your-config-file' to re-upload it.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to target "kubeletconfiguration"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 502.939403ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.
```

// built out the second worker vm node, configured, ready to join the cluster

```
root@dkw2:/home/test# kubeadm join [REDACTED]:6443 --token hp01nf.j8dg6el173973w4e --discovery-token-ca
0b31b7a3c7344e90585135b41f429b7
[preflight] Running pre-flight checks
[WARNING ContainerRuntimeVersion]: You must update your container runtime to a version that support
ing cgroupDriver from kubelet config will be removed in 1.36. For more information, see https://git.k8s.io
tion-over-cri
[preflight] Reading configuration from the "kubeadm-config" ConfigMap in namespace "kube-system"...
[preflight] Use 'kubeadm init phase upload-config kubeadm -vconfig your-config-file' to re-upload it.
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/instance-config.yaml"
[patches] Applied patch of type "application/strategic-merge-patch+json" to target "kubeletconfiguration"
[kubelet-start] Writing kubelet configuration to file "/var/lib/kubelet/config.yaml"
[kubelet-start] Writing kubelet environment file with flags to file "/var/lib/kubelet/kubeadm-flags.env"
[kubelet-start] Starting the kubelet
[kubelet-check] Waiting for a healthy kubelet at http://127.0.0.1:10248/healthz. This can take up to 4m0s
[kubelet-check] The kubelet is healthy after 510.556174ms
[kubelet-start] Waiting for the kubelet to perform the TLS Bootstrap

This node has joined the cluster:
* Certificate signing request was sent to apiservert and a response was received.
* The Kubelet was informed of the new secure connection details.

Run 'kubectl get nodes' on the control-plane to see this node join the cluster.

root@dkw2:/home/test#
```

// verification of joined nodes into cluster from master / control plane vm

```
test@dkm:~$ kubectl get nodes -o wide
```

NAME	STATUS	ROLES	AGE	VERSION	INTERNAL-IP	EXTERNAL-IP
dkm	Ready	control-plane	15h	v1.35.0		<none>
dkw1	Ready	<none>	54m	v1.35.0		<none>
dkw2	Ready	<none>	28s	v1.35.0		<none>

```
test@dkm:~$ |
```

< -- > extended output

OS-IMAGE	KERNEL-VERSION	CONTAINER-RUNTIME
Debian GNU/Linux 13 (trixie)	6.12.63+deb13-amd64	containerd://1.7.24
Debian GNU/Linux 13 (trixie)	6.12.63+deb13-amd64	containerd://1.7.24
Debian GNU/Linux 13 (trixie)	6.12.63+deb13-amd64	containerd://1.7.24

```
test@dkm:~$ kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
dkm	Ready	control-plane	17h	v1.35.0
dkw1	Ready	<none>	3h6m	v1.35.0
dkw2	Ready	<none>	133m	v1.35.0

```
test@dkm:~$ |
```

// showing the pods information

```
test@dkm:~$ kubectl get pods -A
```

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-flannel	kube-flannel-ds-b89j2	1/1	Running	0	3h8m
kube-flannel	kube-flannel-ds-dz7lt	0/1	CrashLoopBackOff	30 (4m53s ago)	134m
kube-flannel	kube-flannel-ds-nmdkx	0/1	CrashLoopBackOff	208 (2m3s ago)	17h
kube-system	coredns-7d764666f9-6cpqj	0/1	ContainerCreating	0	17h
kube-system	coredns-7d764666f9-vvghm	0/1	ContainerCreating	0	17h
kube-system	etcd-dkm	1/1	Running	0	17h
kube-system	kube-apiserver-dkm	1/1	Running	0	17h
kube-system	kube-controller-manager-dkm	1/1	Running	0	17h
kube-system	kube-proxy-fqhbb	0/1	CrashLoopBackOff	24 (5m4s ago)	134m
kube-system	kube-proxy-fwqqx	1/1	Running	0	3h8m
kube-system	kube-proxy-q6rnb	1/1	Running	0	17h
kube-system	kube-scheduler-dkm	1/1	Running	0	17h

```
test@dkm:~$ |
```

// created a proof of concept workload for nginx and a small ai language model

```
test@dkm:~/Documents$ vi proof-concept-workload.yaml
test@dkm:~/Documents$ ls
dkm-config.yaml  proof-concept-workload.yaml
test@dkm:~/Documents$
test@dkm:~/Documents$
test@dkm:~/Documents$ kubectl apply -f proof-concept-workload.yaml
service/nginx-service created
deployment.apps/nginx-deployment created
service/ollama-service created
deployment.apps/ollama-deployment created
test@dkm:~/Documents$
```

```
test@dkm:~$ kubectl get pods -o wide
NAME                                READY   STATUS              RESTARTS
nginx-deployment-59f86b59ff-59vv8   1/1     Running             0
ollama-deployment-764c94bf4c-fjt4n  0/1     ContainerCreating   0
```

< -- > extended output

AGE	IP	NODE	NOMINATED NODE	READINESS GATES
28s	10.244.2.2	dkw2	<none>	<none>
28s	<none>	dkw1	<none>	<none>

// pods progress/logs output

```
test@dkm:~$ kubectl logs nginx-deployment-59f86b59ff-59vv8
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/03 03:02:02 [notice] 1#1: using the "epoll" event method
2026/02/03 03:02:02 [notice] 1#1: nginx/1.29.4
2026/02/03 03:02:02 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/03 03:02:02 [notice] 1#1: OS: Linux 6.12.63+deb13-amd64
2026/02/03 03:02:02 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1073741816:1073741816
2026/02/03 03:02:02 [notice] 1#1: start worker processes
2026/02/03 03:02:02 [notice] 1#1: start worker process 29
2026/02/03 03:02:02 [notice] 1#1: start worker process 30
2026/02/03 03:02:02 [notice] 1#1: start worker process 31
2026/02/03 03:02:02 [notice] 1#1: start worker process 32
test@dkm:~$
```


// pods progress/logs output

```
test@dkm:~$ kubectl logs ollama-deployment-764c94bf4c-fjt4n
Couldn't find '/root/.ollama/id_ed25519'. Generating new private key.
Your new public key is:

ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIGS0uheeZaFvlgL1YoJMYnv6ZPjAq+uFAKm/xLW0und6

time=2026-02-03T03:01:52.260Z level=INFO source=routes.go:1631 msg="server config" env="map[CUDA_VISIBLE_DEVICES: GG
: HIP_VISIBLE_DEVICES: HSA_OVERRIDE_GFX_VERSION: HTTPS_PROXY: HTTP_PROXY: NO_PROXY: OLLAMA_CONTEXT_LENGTH:4096 OLLAMA
OLLAMA_GPU_OVERHEAD:0 OLLAMA_HOST:http://0.0.0.0:11434 OLLAMA_KEEP_ALIVE:5m0s OLLAMA_KV_CACHE_TYPE: OLLAMA_LLM_LIBR
LOADED_MODELS:0 OLLAMA_MAX_QUEUE:512 OLLAMA_MODELS:/mnt/kube-volume OLLAMA_MULTIUSER_CACHE:false OLLAMA_NEW_ENGINE:f
E:false OLLAMA_NUM_PARALLEL:1 OLLAMA_ORIGINS:[http://localhost https://localhost http://localhost:* https://localhos
tp://127.0.0.1:* https://127.0.0.1:* http://0.0.0.0 https://0.0.0.0 http://0.0.0.0:* https://0.0.0.0:* app://* file:
ile://*] OLLAMA_REMOTES:[ollama.com] OLLAMA_SCHED_SPREAD:false OLLAMA_VULKAN:false ROCR_VISIBLE_DEVICES: http_proxy:
time=2026-02-03T03:01:52.267Z level=INFO source=images.go:473 msg="total blobs: 0"
time=2026-02-03T03:01:52.267Z level=INFO source=images.go:480 msg="total unused blobs removed: 0"
time=2026-02-03T03:01:52.268Z level=INFO source=routes.go:1684 msg="Listening on [::]:11434 (version 0.15.4)"
time=2026-02-03T03:01:52.276Z level=INFO source=runner.go:67 msg="discovering available GPUs..."
time=2026-02-03T03:01:52.283Z level=INFO source=server.go:429 msg="starting runner" cmd="/usr/bin/ollama runner --ol
time=2026-02-03T03:01:52.469Z level=INFO source=server.go:429 msg="starting runner" cmd="/usr/bin/ollama runner --ol
time=2026-02-03T03:01:52.579Z level=INFO source=runner.go:106 msg="experimental Vulkan support disabled. To enable,
time=2026-02-03T03:01:52.579Z level=INFO source=types.go:60 msg="inference compute" id=cpu library=cpu compute="" na
ver="" pci_id="" type="" total="7.8 GiB" available="7.7 GiB"
time=2026-02-03T03:01:52.579Z level=INFO source=routes.go:1725 msg="entering low vram mode" "total vram"="0 B" thres
[GIN] 2026/02/03 - 03:01:57 | 200 | 4.767197ms | 127.0.0.1 | HEAD "/"
time=2026-02-03T03:01:58.751Z level=INFO source=download.go:179 msg="downloading 183715c43589 in 10 100 MB part(s)"
time=2026-02-03T03:02:23.182Z level=INFO source=download.go:179 msg="downloading 66b9ea09bd5b in 1 68 B part(s)"
time=2026-02-03T03:02:24.609Z level=INFO source=download.go:179 msg="downloading eb4402837c78 in 1 1.5 KB part(s)"
time=2026-02-03T03:02:26.082Z level=INFO source=download.go:179 msg="downloading 832dd9e00a68 in 1 11 KB part(s)"
time=2026-02-03T03:02:27.520Z level=INFO source=download.go:179 msg="downloading 377ac4d7aeef in 1 487 B part(s)"
[GIN] 2026/02/03 - 03:02:37 | 200 | 40.187792302s | 127.0.0.1 | POST "/api/pull"
test@dkm:~$
```

// pods status

```
test@dkm:~$ kubectl get pods -o wide
NAME                                READY   STATUS    RESTARTS
nginx-deployment-59f86b59ff-59vv8   1/1     Running   0
ollama-deployment-764c94bf4c-fjt4n  1/1     Running   0
```

< -- > extended output

AGE	IP	NODE	NOMINATED NODE	READINESS GATES
5m14s	10.244.2.2	dkw2	<none>	<none>
5m14s	10.244.1.2	dkw1	<none>	<none>

```
test@dkm:~$ kubectl get pods -A
NAMESPACE   NAME                                READY   STATUS    RESTARTS   AGE
default     nginx-deployment-59f86b59ff-59vv8   1/1     Running   0           17m
default     ollama-deployment-764c94bf4c-fjt4n  1/1     Running   0           17m
kube-flannel kube-flannel-ds-9dtdb               1/1     Running   3 (25m ago) 34m
kube-flannel kube-flannel-ds-k5sdx               1/1     Running   7 (28m ago) 34m
kube-flannel kube-flannel-ds-r95g6               1/1     Running   8 (23m ago) 34m
kube-system coredns-7d764666f9-6cpqj           1/1     Running   0           19h
kube-system coredns-7d764666f9-vvvhm           1/1     Running   0           19h
kube-system etcd-dkm                            1/1     Running   1 (20m ago) 19h
kube-system kube-apiserver-dkm         1/1     Running   1 (20m ago) 19h
kube-system kube-controller-manager-dkm 1/1     Running   1 (20m ago) 19h
kube-system kube-proxy-fqhbb           1/1     Running   40 (28m ago) 4h27m
kube-system kube-proxy-fwqgx           1/1     Running   1 (25m ago) 5h21m
kube-system kube-proxy-q6rnb           1/1     Running   1 (20m ago) 19h
kube-system kube-scheduler-dkm         1/1     Running   1 (20m ago) 19h
test@dkm:~$ |
```

// testing the ai language model

```
test@dkw1:~$ curl http://localhost:31434/api/generate -d '{"model": "qwen2.5:1.5b", "prompt": "Why is Kubernetes awesome?", "stream": false}'
{"model": "qwen2.5:1.5b", "created_at": "2026-02-03T03:27:02.763560606Z", "response": "Kubernetes is indeed highly praised for its ability to manage and orchestra
te containerized applications across multiple machines or containers in what is known as a cluster. Here are several reasons why it is often considered \"awe
some\":\n\n1. **Automatic Scaling**: Kubernetes automatically scales up or down the number of instances running an application based on demand, ensuring that
your application can handle traffic spikes without manual intervention.\n\n2. **Resource Management and Balancing**: It manages resources (like CPU, memory)
across nodes in a cluster to ensure optimal utilization of all hardware resources available.\n\n3. **Fault Tolerance**: Kubernetes ensures fault tolerance b
y automatically restarting failed containers or even entire pods when necessary, maintaining the availability of your applications.\n\n4. **Configuration Man
agement**: It handles configuration updates seamlessly, ensuring that changes are applied consistently across all instances running your application.\n\n5. *
*Deployment and Rollbacks**: With its native support for rolling updates and rollbacks, Kubernetes makes it easy to deploy new versions of an application and
revert back if needed without causing downtime.\n\n6. **API and Automation**: The API allows powerful automation capabilities, enabling scripts or commands
that interact with the Kubernetes cluster from any language.\n\n7. **Extensibility**: It's designed in a way that can be extended by developers using custom
resources and controllers, making it highly customizable to meet specific application needs.\n\n8. **Cross-Platform Compatibility**: Kubernetes is compatible
across various operating systems including Linux, macOS, and Windows, making it a versatile choice regardless of the environment where applications are runn
ing.\n\n9. **Monitoring and Logging**: It provides built-in monitoring and logging capabilities, allowing you to track resource usage, container status, and
application performance in real-time.\n\n10. **Community and Ecosystem**: Kubernetes has a large community and robust ecosystem with countless plugins and ad
d-ons that make it incredibly powerful for handling more advanced use cases beyond just container management.\n\nThese features make Kubernetes indispensable
for managing modern microservices architectures efficiently across distributed systems.", "done": true, "done_reason": "stop", "context": [151644, 8948, 198, 2610, 52
5, 1207, 16948, 11, 3465, 553, 54364, 14817, 13, 1446, 525, 264, 10950, 17847, 13, 151645, 198, 151644, 872, 198, 10234, 374, 66374, 12456, 30, 151645, 198, 151644, 77091, 198, 42, 29827, 3
74, 12824, 7548, 36375, 369, 1181, 5726, 311, 10091, 323, 69884, 7698, 5476, 1506, 8357, 3941, 5248, 12645, 476, 23853, 304, 1128, 374, 3881, 438, 264, 10652, 13, 5692, 525, 3807, 7966, 317
0, 432, 374, 3545, 6509, 330, 16875, 51418, 16, 13, 3070, 62790, 88001, 95518, 66374, 9463, 28405, 705, 476, 1495, 279, 1372, 315, 13121, 4303, 458, 3766, 3118, 389, 7479, 11, 22573, 429, 69
7, 3766, 646, 3705, 9442, 50660, 2041, 11376, 20949, 382, 17, 13, 3070, 4783, 9551, 323, 29420, 8974, 95518, 1084, 28872, 4963, 320, 4803, 13940, 11, 4938, 8, 3941, 7798, 304, 264, 10652, 31
1, 5978, 22823, 49449, 315, 678, 11773, 4963, 2500, 382, 18, 13, 3070, 58780, 350, 31661, 95518, 66374, 25351, 14527, 24098, 553, 9463, 92524, 4641, 23853, 476, 1496, 4453, 54587, 979, 587
1, 11, 20337, 279, 18048, 315, 697, 8357, 382, 19, 13, 3070, 7688, 9551, 95518, 1084, 13469, 6546, 8837, 60340, 11, 22573, 429, 4344, 525, 9251, 20699, 3941, 678, 13121, 4303, 697, 3766, 382
, 20, 13, 3070, 75286, 323, 14686, 24113, 95518, 3085, 1181, 9867, 1824, 369, 20097, 8837, 323, 6502, 24113, 11, 66374, 3643, 432, 4135, 311, 10517, 501, 10795, 315, 458, 3766, 323, 41128, 1
182, 421, 4362, 2041, 14381, 74854, 382, 21, 13, 3070, 7082, 323, 53778, 95518, 576, 5333, 6147, 7988, 32662, 16928, 11, 27362, 19502, 476, 11293, 429, 16282, 448, 279, 66374, 10652, 504, 8
94, 4128, 382, 22, 13, 3070, 6756, 724, 3147, 95518, 1084, 748, 6188, 304, 264, 1616, 429, 646, 387, 11577, 553, 13402, 1667, 2526, 4963, 323, 26225, 11, 3259, 432, 7548, 62074, 311, 3367, 31
51, 3766, 3880, 382, 23, 13, 3070, 28501, 12, 17296, 77995, 95518, 66374, 374, 18146, 3941, 5257, 10350, 5942, 2670, 14340, 11, 67178, 11, 323, 5515, 11, 3259, 432, 264, 31945, 5754, 15484,
315, 279, 4573, 1380, 8357, 525, 4303, 382, 24, 13, 3070, 98062, 323, 42706, 95518, 1084, 5707, 5798, 3419, 16558, 323, 8392, 16928, 11, 10693, 498, 311, 3754, 5101, 10431, 11, 5476, 2639, 1
1, 323, 3766, 5068, 304, 1931, 7246, 382, 16, 15, 13, 3070, 33768, 323, 468, 23287, 95518, 66374, 702, 264, 3460, 3942, 323, 21765, 24982, 448, 27601, 17215, 323, 912, 59126, 429, 1281, 432,
16815, 7988, 369, 11589, 803, 10847, 990, 5048, 7797, 1101, 5476, 6240, 382, 9485, 4419, 1281, 66374, 63184, 369, 18150, 6481, 8003, 12779, 77235, 29720, 3941, 4237, 5942, 13], "total_du
ration": 203376961112, "load_duration": 2453194042, "prompt_eval_count": 34, "prompt_eval_duration": 11963557124, "eval_count": 393, "eval_duration": 187594310963}test@
dkw1:~$
```

// expanded view/cleaned up response for readability below:



// Cleaned up response for readability:

"response": "Kubernetes is indeed highly praised for its ability to manage and orchestrate containerized applications across multiple machines or containers in what is known as a cluster. Here are several reasons why it is often considered "awesome"

****Automatic Scaling****: Kubernetes automatically scales up or down the number of instances running an application based on demand, ensuring that your application can handle traffic spikes without manual intervention.

****Resource Management and Balancing****: It manages resources (like CPU, memory) across nodes in a cluster to ensure optimal utilization of all hardware resources available.

****Fault Tolerance****: Kubernetes ensures fault tolerance by automatically restarting failed containers or even entire pods when necessary, maintaining the availability of your applications.

****Configuration Management****: It handles configuration updates seamlessly, ensuring that changes are applied consistently across all instances running your application.

****Deployment and Rollbacks****: With its native support for rolling updates and rollbacks, Kubernetes makes it easy to deploy new versions of an application and revert back if needed without causing downtime.

****API and Automation****: The API allows powerful automation capabilities, enabling scripts or commands that interact with the Kubernetes cluster from any language.

****Extensibility****: It's designed in a way that can be extended by developers using custom resources and controllers, making it highly customizable to meet specific application needs.\n\n8.

****Cross-Platform Compatibility****: Kubernetes is compatible across various operating systems including Linux, macOS, and Windows, making it a versatile choice regardless of the environment where applications are running.

****Monitoring and Logging**:** It provides built-in monitoring and logging capabilities, allowing you to track resource usage, container status, and application performance in real-time.

****Community and Ecosystem**:** Kubernetes has a large community and robust ecosystem with countless plugins and add-ons that make it incredibly powerful for handling more advanced use cases beyond just container management. These features make Kubernetes indispensable for managing modern microservices architectures efficiently across distributed systems.

```
", "done": true, "done_reason": "stop",
```

```
// second test prompt for the ai language model
```

```
test@dkw1:~$ curl http://localhost:31434/api/generate -d '{"model": "qwen2.5:1.5b", "prompt": "Give me a paragraph or two  
victory of wrangling this kubernetes cluster project", "stream": false}'  
{ "model": "qwen2.5:1.5b", "created_at": "2026-02-03T03:41:50.982605847Z",
```

```
"response": "
```

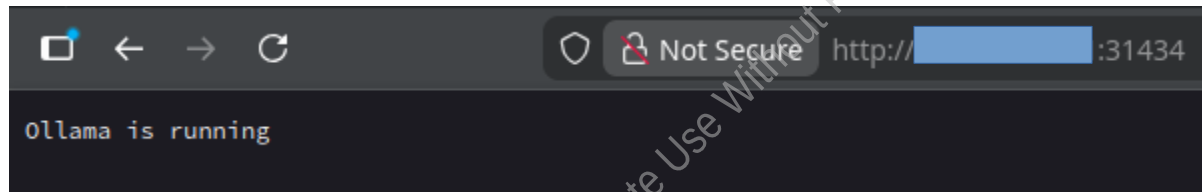
The key to successfully wrangling Kubernetes cluster projects involves several critical steps and considerations. First, ensuring that your infrastructure is properly set up with the right version control tools such as Git for source management, Jenkins for CI/CD pipeline setup, and Rancher or equivalent orchestrators for managing applications across multiple nodes. Secondly, mastering the art of automation through scripting languages like Bash, YAML, and kubectl scripts can significantly streamline the deployment process, ensuring that every Kubernetes node is correctly configured without manual intervention. This includes understanding how to create, update, and delete deployments, services, stateful sets, and more, which are all part of a typical Kubernetes project. Furthermore, leveraging monitoring tools such as Prometheus or Grafana for log analysis can help in identifying and resolving issues faster by providing real-time insights into the performance and health of your application. This proactive approach to monitoring is crucial for maintaining uptime and reliability in your service-oriented architecture. Lastly, maintaining documentation practices ensures that all team members are aligned on project goals and processes, which is essential for avoiding misunderstandings and ensuring consistency across different teams working on the same Kubernetes cluster projects. Regular code reviews and pair programming can also help improve collaboration and quality of work among developers.

```
", "done": true, "done_reason": "stop", "context"
```

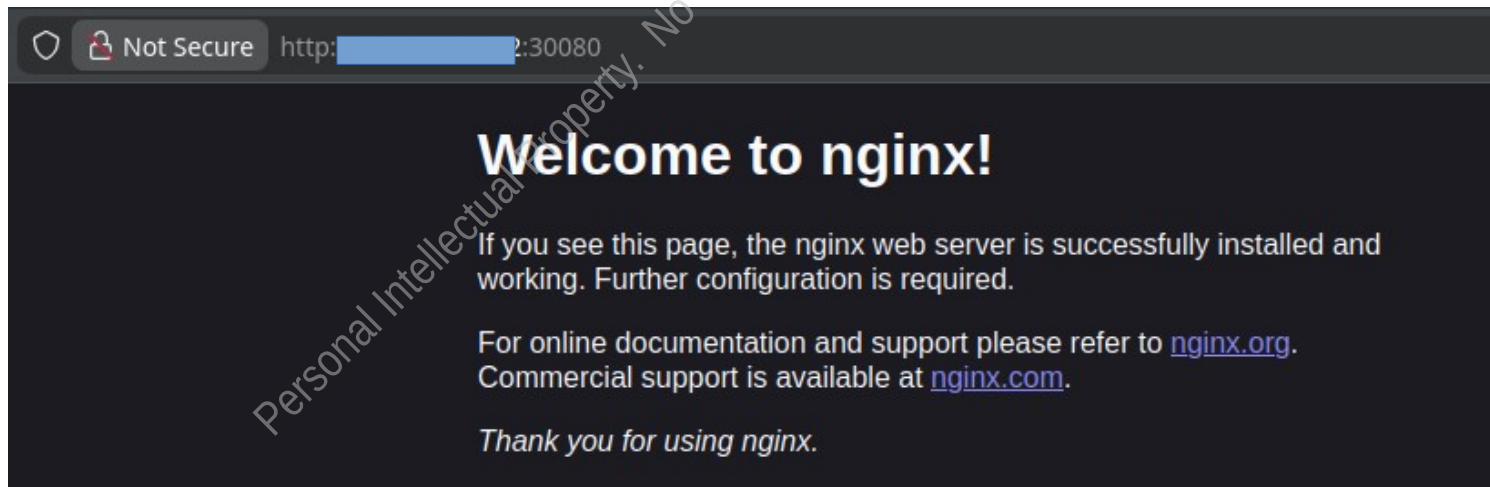

// testing the rest

```
test@dkm:~$ kubectl get svc
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
kubernetes           ClusterIP      10.96.0.1        <none>            443/TCP          20h
nginx-service         NodePort       10.97.235.170    <none>            80:30080/TCP     60m
ollama-service        NodePort       10.98.245.31     <none>            11434:31434/TCP  60m
test@dkm:~$
```

// ollama verification, via vm ip and port : 31434



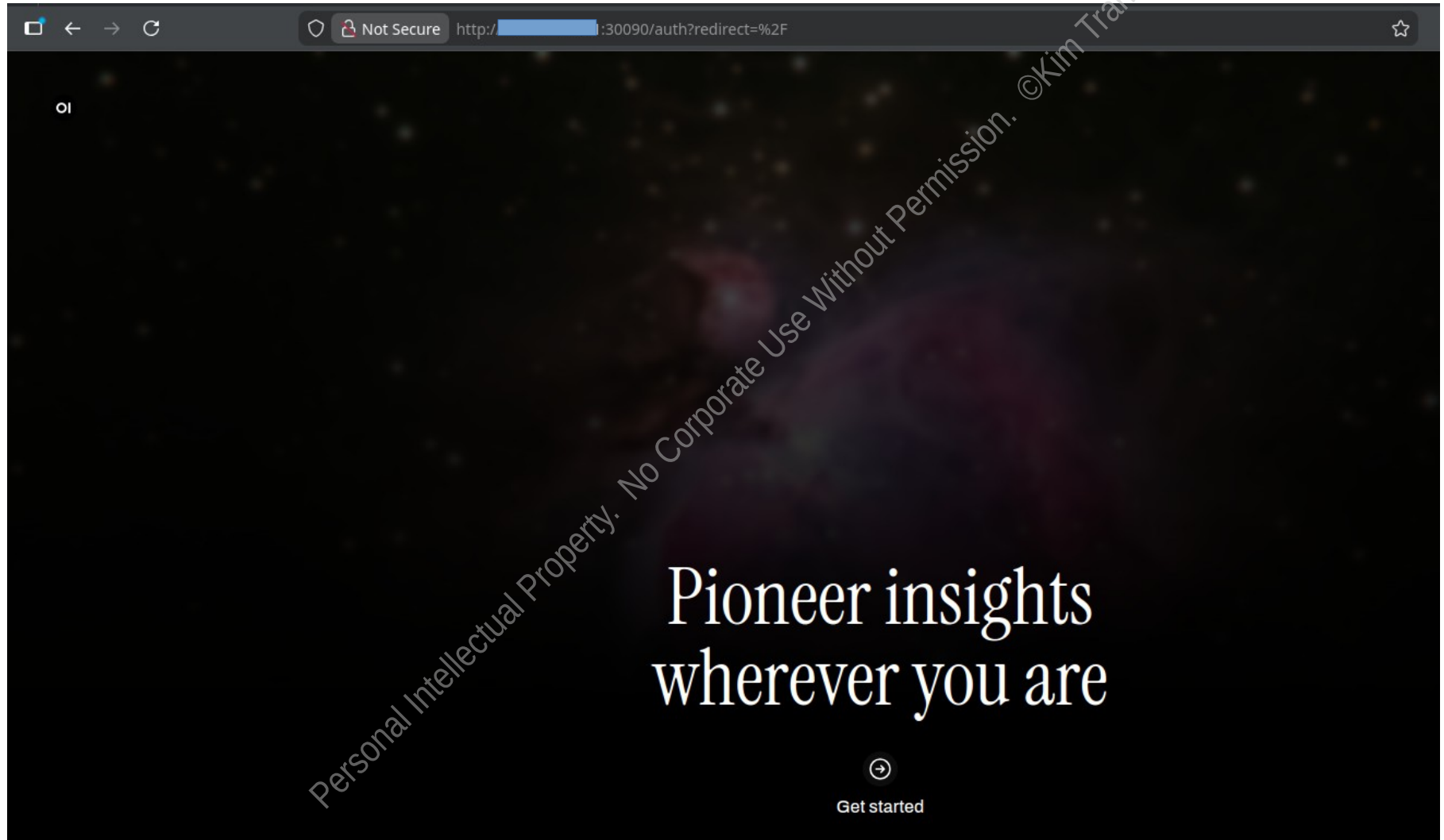
// nginx verification, via vm ip and port : 30080



// adding an UI for the ollama ai language model

```
test@dkm:~$ vi Documents/ollama-ai-llm-gui.yaml
test@dkm:~$
test@dkm:~$
test@dkm:~$
test@dkm:~$
test@dkm:~$
test@dkm:~$
test@dkm:~$
test@dkm:~$
test@dkm:~$ kubectl apply -f Documents/ollama-ai-llm-gui.yaml
deployment.apps/open-webui unchanged
service/open-webui-service created
test@dkm:~$
```


// web gui for the ai language model



// continued

A screenshot of a web browser displaying the Open WebUI installation page. The browser's address bar shows a URL starting with 'http://' and a port number, indicating a local development environment. The page has a dark theme with a black background. In the top left corner, there is a small circular logo with the letters 'OI'. The main heading is 'Get started with Open WebUI' in white. Below the heading, a note states: 'Open WebUI does not make any external connections, and your data stays securely on your locally hosted server.' There are three input fields: 'Name' with the placeholder 'Enter Your Full Name', 'Email' with the placeholder 'Enter Your Email', and 'Password' with the placeholder 'Enter Your Password' and a toggle icon for password visibility. A large, rounded rectangular button labeled 'Create Admin Account' is positioned at the bottom of the form area. A large, diagonal watermark is overlaid across the entire page, reading 'Personal Intellectual Property. No Corporate Use Without Permission. © Kim Tran'.

Lab afterthoughts and lessons

This was a grueling 48 hours of creating and learning and debugging everything that did not go smoothly.

This was not follow a tutorial and everything should work as expected.

It was a start with the basics of what I knew, such as creating the vm's and knowing the core components to be installed, such as containerd, kubeadm, kubect, kubelet.

Everything else was followed by basic initialization of the services, and troubleshooting the pod networks, from flannel, to the services not executing. Permissions not correct on certain directories and files, to errors with the containers/pods not deploying, checking the various logs and iteration debugging, rather than massive shotgun approaches, change and test, restore and test.

Great learning to get the brain in gear, but everything was all over the place with so many cross disciplines, the few hours of sleep in between the sessions to get this up and running. At times it felt like just being a cowboy and this deployment of the cluster was half baked, of it working, but not fully working, some services showed up, but the integrity of those services were questionable and not talking to the other nodes and it was just chaos.

Rebuilt this test cluster a few times to get into the flow of things and it was quit satisfying to see the results. :-)

==> MY ELECTRICITY BILL IS THE BATTLE SCAR OF THIS KUBERNETES CLUSTER PROOF OF CONCEPT ;-)<==

-real hardware, real cost, real pain, real learning, real experience

Personal Intellectual Property. No Corporate Use Without Permission.

©Kim Tran

Home half rack of equipment/gear connected in the dark
pretty leds :-D ^_^



Personal Intellectual Property. No Corporate Use Without Permission.

©Kim Tran